Especificación de Arquitectura Abierta

Continuación del Paquete Técnico: Sistema de Video Neuronal Implícito Descentralizado (SIREN-VDS)

Objetivo del documento: Proveer la especificación matemática, el pipeline de procesamiento y la arquitectura de síntesis en tiempo de ejecución para laboratorios tecnológicos, colectivos de código abierto e instituciones sin fines de lucro a nivel global.

1. Fundamentos Matemáticos y Estructura del Modelo (SIREN-MLP)

- **Mapeo de coordenadas espacio-temporales:** Cada fotograma del video no se almacena como matriz de píxeles, sino como una función continua parametrizada $f: \mathbb{R}^3 o \mathbb{R}^3$ que mapea coordenadas espacio-temporales (x, y, t) directamente a valores de color RGB.
- **Capas con activaciones de seno:** A diferencia de las redes tradicionales con activaciones ReLU o sigmoides (que sufren de pérdida de detalle en derivadas de orden superior), la arquitectura emplea funciones sinusoidales puras, esenciales para representar señales continuas y sus derivadas espaciales:

$$\Phi_1(x) = \sin(\mathbf{W}_1 x + \mathbf{b}_1)$$

- **Inicialización especial de pesos (Periodic Scale):** Para evitar la atenuación de gradientes en capas profundas con activaciones seno, la primera capa se inicializa con una distribución uniforme escalada por un factor de frecuencia ω_0 (típicamente ~ 30), mientras que las capas ocultas posteriores se escalan con $\sqrt{6 / n}$, donde n es la dimensión de entrada de la capa.
- **Optimización por Descenso de Gradiente:** El entrenamiento ajusta los pesos de la red minimizando el Error Cuadrático Medio ($ext{MSE}$) entre el color predicho y el fotograma fuente real muestreado:

$$\mathcal{L} = rac{1}{N} \sum \left(f(x, y, t) - ext{Color}_{ ext{real}} ight)^2$$

2. Pipeline de Procesamiento y Extracción de Datos

- **Ingestión y Muestreo de Píxeles:** En lugar de decodificar flujos completos mediante códecs propietarios pesados, el sistema lee secuencias de fotogramas mediante rutinas ligeras (como extracción directa de matrices de píxeles), generando un conjunto de datos plano de tensores de entrenamiento compuestos por tuplas de coordenadas y canales de color:

$$ext{Dataset} = \{ (x_i, y_i, t_i), (R_i, G_i, B_i) \}_{i=1}^N$$

- **Optimización en Botes (Mini-batches):** Los datos se procesan en bloques aleatorios para ajustar los parámetros de la red mediante retropropagación (*backpropagation*) escrita en bajo nivel con NumPy puro.

3. Motor de Síntesis en Tiempo de Ejecución (On-The-Fly)

- **Ausencia de decodificación de fotogramas:** La reproducción no requiere búferes de lectura de video masivos ni decodificadores de hardware comerciales; la aplicación genera el fotograma actual evaluando matricialmente la red para una malla de coordenadas constante en un tiempo t específico:

$$ext{Frame}_{t} = ext{SirenMLP}(ext{Grid}_{xy}, t)$$

- **Aceleración por Shaders de Hardware (OpenGL ES / Vulkan):** Para superar el cuello de botella de la ejecución en CPU, las operaciones matriciales de las capas de la red se compilan en *Compute Shaders* ejecutados directamente en la unidad de procesamiento gráfico móvil, paralelizando el cálculo de los senos por cada píxel de la pantalla.

4. Protocolo de Intercambio y Formato de Archivos Descentralizado

- **Serialización de Pesos Netos:** El video completo queda condensado en un único archivo de metadatos y tensores de pesos con extensión abierta (ej. ***.siren***), cuyo tamaño oscila típicamente entre cientos de kilobytes y pocos megabytes.
- **Independencia de infraestructura centralizada:** Al ser archivos diminutos, la distribución se efectúa mediante protocolos peer-to-peer (P2P) o repositorios distribuidos sin requerir pasarelas de pago ni servidores de streaming multimillonarios.